```
from google.colab import drive
drive.mount('/content/drive')
d = '/content/drive/MyDrive/4thSem-PES/Code/Phishing-main/'
```

```
Mounted at /content/drive
```

```
import warnings
warnings.filterwarnings('ignore')
```

```
#  imports all the necessary libraries for data manipulation, machine learning models, and evaluation metrics.
# It includes popular libraries like pandas, numpy, scikit-learn, imblearn, xgboost, and lightgbm.
import pandas as pd # For data manipulation and analysis, especially with DataFrames
import numpy as np # For numerical operations, especially with arrays
import itertools # For creating iterators for efficient looping

from sklearn.preprocessing import * # For data preprocessing techniques like scaling and encoding
from sklearn.model_selection import * # For splitting data, cross-validation, and hyperparameter tuning
from sklearn.metrics import * # For evaluating model performance with various metrics
from imblearn.metrics import * # For evaluation metrics specifically designed for imbalanced datasets

from sklearn.linear_model import LogisticRegression # A linear model for binary classification
from sklearn.neighbors import KNeighborsClassifier # A non-parametric classification algorithm
from sklearn.svm import SVC # A powerful and versatile classification algorithm
from sklearn.naive_bayes import GaussianNB # A probabilistic classifier based on Bayes' theorem
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis # A linear classifier for dimensionality red

from sklearn.tree import DecisionTreeClassifier # A tree-based model for classification
from sklearn.ensemble import RandomForestClassifier # An ensemble method using multiple decision trees
from sklearn.ensemble import ExtraTreesClassifier # Another ensemble method similar to Random Forest
from sklearn.ensemble import AdaBoostClassifier # An ensemble method that combines multiple weak learners
from sklearn.ensemble import GradientBoostingClassifier # An ensemble method that builds trees sequentially

from sklearn.ensemble import StackingClassifier # For combining multiple models in a stacking ensemble
from mlxtend.classifier import StackingCVClassifier # A more robust stacking classifier with cross-validation

from sklearn.neural_network import MLPClassifier # A feedforward artificial neural network classifier


from xgboost import XGBClassifier # An optimized distributed gradient boosting library
from lightgbm import LGBMClassifier # A gradient boosting framework that uses tree-based learning algorithms

import matplotlib.pyplot as plt # For creating static, animated, and interactive visualizations
```

```
#  loads the dataset from a CSV file named 'mendeley_20.csv' into a pandas DataFrame named 'data'.
# It then displays the DataFrame to show its initial structure and content.
data= pd.read_csv(d + 'mendeley_20.csv')
data
```

	qty_dot_url	qty_hyphen_url	qty_underline_url	qty_slash_url	qty_questionmark_url	qty_equal_url	qty_at_u
0	3	0	0	1	0	0	
1	5	0	1	3	0	3	
2	2	0	0	1	0	0	
3	4	0	2	5	0	0	
4	2	0	0	0	0	0	
...	...	...	...	...	...	...	
88642	3	1	0	0	0	0	
88643	2	0	0	0	0	0	
88644	2	1	0	5	0	0	
88645	2	0	0	1	0	0	
88646	2	0	0	0	0	0	

88647 rows × 112 columns

```
#  separates the features (independent variables) from the target variable.
# It creates a new DataFrame 'x' by dropping the 'phishing' column from the 'data' DataFrame.
# The 'phishing' column is assumed to be the target variable indicating whether a website is phishing or not.
x= data.drop(['phishing'],axis=1)
x
```

	qty_dot_url	qty_hyphen_url	qty_underline_url	qty_slash_url	qty_questionmark_url	qty_equal_url	qty_at_u
0	3	0	0	1	0	0	
1	5	0	1	3	0	3	
2	2	0	0	1	0	0	
3	4	0	2	5	0	0	
4	2	0	0	0	0	0	
...	...	...	...	...	...	...	
88642	3	1	0	0	0	0	
88643	2	0	0	0	0	0	
88644	2	1	0	5	0	0	
88645	2	0	0	1	0	0	
88646	2	0	0	0	0	0	

88647 rows × 111 columns

```
#  extracts the target variable (dependent variable) from the 'data' DataFrame.
# The 'phishing' column is assigned to a new Series named 'y'.
y=data['phishing']
y
```

```
0        1
1        1
2        0
3        1
4        0
       ..
88642    0
88643    0
88644    1
88645    1
88646    0
Name: phishing, Length: 88647, dtype: int64
```

```
# For demonstration purposes, the entire dataset 'x' is assigned as the training features 'x_train',
# and the entire target variable 'y' is assigned as the training labels 'y_train'.
# In a typical machine learning workflow, the data would be split into training and testing sets.
x_train=x
y_train=y
```

```
#  calculates and displays the count of each unique value in the target variable 'y'.
# This is useful to check for class imbalance, where one class has significantly more samples than others.
y.value_counts()
```

```
0    58000
1    30647
Name: phishing, dtype: int64
```

defining models

```
#  initializes various machine learning classifiers with specific hyperparameters.
# These classifiers include K-Nearest Neighbors (knn), Support Vector Machine (svm), Logistic Regression (lr),
# Random Forest (rf), AdaBoost (ada), Extra Trees (extra), LightGBM (lgbm), Gradient Boosting (grad),
# XGBoost (xg), and a Multi-layer Perceptron (nn).
knn= KNeighborsClassifier(n_neighbors=10,n_jobs=-1)
svm= SVC(random_state=10,kernel='rbf')
lr = LogisticRegression()

rf= RandomForestClassifier(n_jobs=-1,random_state=10)
ada= AdaBoostClassifier(random_state=100)
extra = ExtraTreesClassifier(n_jobs=-1,random_state=10)

lgbm = LGBMClassifier(n_jobs=-1,random_state=10)
grad = GradientBoostingClassifier()
xg = XGBClassifier(use_label_encoder =False, eval_metric='logloss')

nn= MLPClassifier(activation = "relu", alpha = 0.001, hidden_layer_sizes = (32,32,32), random_state=10)
```

```
#  creates a dictionary named 'classifiers' to store several initialized ensemble and neural network models.
# This makes it easier to reference and iterate through these specific models later.
```

```
classifiers= {
    "RandomForest": rf,
    "ExtraTree": extra,
    "gradBoost": grad,
    "adaboost": ada,
    "LGBM": lgbm,
    "xgb": xg,
    "nn":nn}
```

```
#  assigns the initialized Multi-layer Perceptron (nn) classifier to a variable named 'classifier_2'.
# This might be for a specific purpose where only one classifier is needed at a time or for a subsequent stackin
classifier_2 = nn
```

```
#  defines a dictionary named 'scores' containing various scoring metrics wrapped with 'make_scorer'.
# These metrics (accuracy, recall, precision, geometric mean, f1-score, and ROC AUC) will be used to evaluate th
scores={'accuracy': make_scorer(accuracy_score),
        'recall' : make_scorer(recall_score),
        'precision':make_scorer(precision_score),
        'gmean': make_scorer(geometric_mean_score),
        'f1':make_scorer(f1_score),
        'roc': make_scorer(roc_auc_score)
        }
```

base models

```
#  performs 10-fold cross-validation on the Random Forest classifier (rf) using the defined scoring metrics.
# It then calculates and displays the mean of the cross-validation results for each metric.
rf_result = cross_validate(rf, x_train, y_train, cv = 10, scoring=scores)
df= pd.DataFrame(rf_result)
df.mean(axis=0)
```

```
fit_time          2.725133
score_time        0.041724
test_accuracy     0.971663
test_recall       0.961301
test_precision    0.956945
test_gmean        0.969185
test_f1           0.959111
test_roc          0.969220
dtype: float64
```

```
#  performs 10-fold cross-validation on the AdaBoost classifier (ada) using the defined scoring metrics.
# It then calculates and displays the mean of the cross-validation results for each metric.
ada_result= cross_validate(ada, x_train, y_train, cv = 10, scoring=scores)
df= pd.DataFrame(ada_result)
df.mean(axis=0)
```

```
fit_time          5.876793
score_time        0.101128
test_accuracy     0.936490
test_recall       0.907104
test_precision    0.909045
test_gmean        0.929284
test_f1           0.908056
test_roc          0.929561
dtype: float64
```

```
#  performs 10-fold cross-validation on the Extra Trees classifier (extra) using the defined scoring metrics.
# It then calculates and displays the mean of the cross-validation results for each metric.
extra_result= cross_validate(extra, x_train, y_train, cv = 10, scoring=scores)
df= pd.DataFrame(extra_result)
df.mean(axis=0)
```

```
fit_time          2.776407
score_time        0.045096
test_accuracy     0.970918
test_recall       0.959311
test_precision    0.956692
test_gmean        0.968139
test_f1           0.957996
test_roc          0.968181
dtype: float64
```

```
#  performs 10-fold cross-validation on the Gradient Boosting classifier (grad) using the defined scoring metric
# It then calculates and displays the mean of the cross-validation results for each metric.
grad_result= cross_validate(grad, x_train, y_train, cv = 10, scoring=scores)
df= pd.DataFrame(grad_result)
df.mean(axis=0)
```

```
fit_time          22.501707
score_time         0.028397
test_accuracy      0.953478
test_recall        0.934610
test_precision     0.931095
test_gmean         0.948918
test_f1            0.932845
test_roc           0.949029
dtype: float64
```

```
#  performs 10-fold cross-validation on the LightGBM classifier (lgbm) using the defined scoring metrics.
# It then calculates and displays the mean of the cross-validation results for each metric.
lgbm_result= cross_validate(lgbm, x_train, y_train, cv = 10, scoring=scores, n_jobs=-1)
df= pd.DataFrame(lgbm_result)
df.mean(axis=0)
```

```
fit_time          3.501818
score_time        0.079918
test_accuracy     0.966812
test_recall       0.953274
test_precision    0.950870
test_gmean        0.963563
test_f1           0.952066
test_roc          0.963620
dtype: float64
```

```
#  performs 10-fold cross-validation on the XGBoost classifier (xg) using the defined scoring metrics.
# It then calculates and displays the mean of the cross-validation results for each metric.
xg_result= cross_validate(xg, x_train, y, cv = 10, scoring=scores, n_jobs=-1)
df= pd.DataFrame(xg_result)
df.mean(axis=0)
```

```
fit_time          28.009947
score_time         0.040733
test_accuracy      0.970828
test_recall        0.957777
test_precision     0.957863
test_gmean         0.967697
test_f1            0.957812
test_roc           0.967751
dtype: float64
```

```
#  performs 10-fold cross-validation on the Multi-layer Perceptron classifier (nn) using the defined scoring met
# It then calculates and displays the mean of the cross-validation results for each metric.
nn_result= cross_validate(nn, x_train, y, cv = 10, scoring=scores,  n_jobs=-1)
df= pd.DataFrame(nn_result)
df.mean(axis=0)
```

```
fit_time          62.715342
score_time         0.037506
test_accuracy      0.921058
test_recall        0.897839
test_precision     0.878532
test_gmean         0.914952
test_f1            0.886950
test_roc           0.915583
dtype: float64
```

```
#  initializes a Gaussian Naive Bayes classifier (nb).
# It then performs 10-fold cross-validation on the Naive Bayes classifier using the defined scoring metrics,
# and calculates and displays the mean of the cross-validation results for each metric.
nb= GaussianNB()
nb_result= cross_validate(nb, x_train, y, cv = 10, scoring=scores)
df= pd.DataFrame(nb_result)
df.mean(axis=0)
```

```
fit_time          0.169213
score_time        0.023905
test_accuracy     0.842860
test_recall       0.628055
test_precision    0.883814
test_gmean        0.774996
test_f1           0.734265
test_roc          0.792209
dtype: float64
```

```
#  performs 10-fold cross-validation on the K-Nearest Neighbors classifier (knn) using the defined scoring metri
# It then calculates and displays the mean of the cross-validation results for each metric.
knn_result= cross_validate(knn, x_train, y, cv = 10, scoring=scores, n_jobs=-1)
df= pd.DataFrame(knn_result)
df.mean(axis=0)
```

```
fit_time          0.057712
score_time       16.391473
```

```
test_accuracy      0.866437
test_recall        0.763534
test_precision     0.835934
test_gmean         0.838485
test_f1            0.798083
test_roc           0.842172
dtype: float64
```

```
#  performs 10-fold cross-validation on the Support Vector Machine classifier (svm) using the defined scoring me
# It then calculates and displays the mean of the cross-validation results for each metric.
svm_result= cross_validate(svm, x_train, y, cv = 10, scoring=scores, n_jobs=-1)
df= pd.DataFrame(svm_result)
df.mean(axis=0)
```

```
fit_time          1845.801982
score_time          92.756810
test_accuracy        0.757984
test_recall          0.567266
test_precision       0.679751
test_gmean           0.697948
test_f1              0.618419
test_roc             0.713012
dtype: float64
```

```
#  performs 10-fold cross-validation on the Logistic Regression classifier (lr) using the defined scoring metric
# It then calculates and displays the mean of the cross-validation results for each metric.
lr_result= cross_validate(lr, x_train, y, cv = 10, scoring=scores,  n_jobs=-1)
df= pd.DataFrame(lr_result)
df.mean(axis=0)
```

```
fit_time          8.901156
score_time        0.021805
test_accuracy     0.893353
test_recall       0.797275
test_precision    0.883447
test_gmean        0.866992
test_f1           0.836984
test_roc          0.870698
dtype: float64
```

stacking - proba

```
#  sets up a StackingCVClassifier using a predefined list of base classifiers (sel_classifier) and a Multi-layer
# It is configured to use predicted probabilities ('use_probas = True') from the base classifiers as input to th
# Finally, it performs 10-fold cross-validation on this stacking model and stores the results.

stacking_result = []
stacking_names= []

nn_meta= MLPClassifier(activation = "relu", alpha = 0.001, hidden_layer_sizes = (6,12,24,12,6), random_state=10)

sel_classifier= [rf, ada, extra, lgbm, grad, xg, nn]

scl = StackingCVClassifier(classifiers= sel_classifier,
                           meta_classifier = nn_meta, # use meta-classifier
                           use_probas = True,   # use_probas = True/False
                           random_state = 10)

st_result= cross_validate(scl, x_train, y, cv = 10, scoring=scores,n_jobs=-1)
```

```
#  converts the cross-validation results from the StackingCVClassifier (st_result) into a pandas DataFrame.
# It then calculates and displays the mean of all the evaluation metrics across the 10 folds, providing an overa
df= pd.DataFrame(st_result)
df.mean(axis=0)
```

```
fit_time          21.672631
score_time         0.083033
test_accuracy      0.974759
test_recall        0.984243
test_precision     0.971064
test_gmean         0.973425
test_f1            0.977543
test_roc           0.973541
dtype: float64
```

```
#  performs a GridSearchCV to optimize the hyperparameters of the meta-classifier within the StackingCVClassifie
# It defines a parameter grid for 'hidden_layer_sizes' of the MLPClassifier (nn_basic) used as the meta-classifi
# The GridSearchCV will systematically train and evaluate the StackingCVClassifier with each combination of hype
sel_classifier= [rf, ada, extra, lgbm, grad, xg, nn]
```

```
nn_basic = MLPClassifier(random_state=10)

scl = StackingCVClassifier(classifiers= sel_classifier,
                           meta_classifier = nn_basic, # use meta-classifier
                           use_probas = True,   # use_probas = True/False
                           random_state = 10)

params= {
    'meta_classifier__hidden_layer_sizes': [(12,12,12),(12,6,6),(6,12,32,12,6),(12,24,12,6)],
}

grid = GridSearchCV(estimator=scl,
                    param_grid=params,
                    cv=10,
                    refit=True,
                    n_jobs=-1)

grid.fit(x_train, y_train)
```

```
GridSearchCV(cv=10,
             estimator=StackingCVClassifier(classifiers=[RandomForestClassifier(n_jobs=-1,
                                                                                random_state=10),
                                                         AdaBoostClassifier(random_state=100),
                                                         ExtraTreesClassifier(n_jobs=-1,
                                                                              random_state=10),
                                                         LGBMClassifier(random_state=10),
                                                         GradientBoostingClassifier(),
                                                         XGBClassifier(base_score=None,
                                                                       booster=None,
                                                                       colsample_bylevel=None,
                                                                       colsample_bynode=None,
                                                                       col...
                                                                       subsample=None,
                                                                       tree_method=None,
                                                                       use_label_encoder=False,
                                                                       validate_parameters=None,
                                                                       verbosity=None),
                                                         MLPClassifier(alpha=0.001,
                                                                       hidden_layer_sizes=(32,
                                                                                           32,
                                                                                           32),
                                                                       random_state=10)],
                                                    meta_classifier=MLPClassifier(random_state=10),
                                                    random_state=10, use_probas=True),
             n_jobs=-1,
             param_grid={'meta_classifier__hidden_layer_sizes': [(12, 12, 12),
                                                                 (12, 6, 6),
                                                                 (6, 12, 32, 12,
                                                                  6),
                                                                 (12, 24, 12,
                                                                  6)]})
```

```
#  converts the results of the GridSearchCV into a pandas DataFrame.
# It then sorts the results by 'rank_test_score' to easily identify the best performing hyperparameter combinati
df= pd.DataFrame(grid.cv_results_)
df.sort_values(by='rank_test_score')
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_meta_classifier__hidden_layer_sizes	
1	897.227048	86.170097	1.450027	0.402412	(12, 6, 6)	{'meta_c
3	789.143794	172.485958	0.783166	0.869165	(12, 24, 12, 6)	{'meta_c
0	960.579238	122.727922	1.266486	0.566797	(12, 12, 12)	{'meta_c
2	1030.604501	66.966603	1.008628	0.472403	(6, 12, 32, 12, 6)	{'meta_c

feature selection

```
from sklearn.feature_selection import RFECV
```

```
#  initializes an RFECV (Recursive Feature Elimination with Cross-Validation) selector with a Random Forest clas
# It then fits the selector to the entire dataset (x, y) and extracts the indices of the features ranked as opti
selector = RFECV(rf, cv=3, n_jobs=-1)
selector = selector.fit(x, y)
rf_col,= np.where(selector.ranking_==1)
rf_col
```

```
array([  0,   1,   3,  18,  19,  20,  36,  37,  40,  41,  42,  43,  45,
        46,  48,  50,  51,  52,  56,  57,  58,  59,  60,  64,  65,  67,
        69,  70,  74,  75,  76,  83,  93,  95,  97,  98,  99, 100, 101,
       102, 103, 104, 105, 106, 107], dtype=int64)
```

```
#  initializes an RFECV selector with an AdaBoost classifier (ada).
# It then fits the selector to the dataset and extracts the indices of the features ranked as optimal (i.e., ran
selector = RFECV(ada, cv=3, n_jobs=-1)
selector = selector.fit(x, y)
ada_col,= np.where(selector.ranking_==1)
ada_col
```

```
array([ 17,  18,  19,  20,  37,  41,  57,  60,  75,  93,  97,  98,  99,
       100, 102, 103, 105, 106, 110], dtype=int64)
```

```
#  initializes an RFECV selector with an Extra Trees classifier (extra).
# It then fits the selector to the dataset and extracts the indices of the features ranked as optimal (i.e., ran
selector = RFECV(extra, cv=3, n_jobs=-1)
selector = selector.fit(x, y)
extra_col= np.where(selector.ranking_==1)
extra_col
```

```
(array([  0,   1,   3,  18,  19,  36,  37,  43,  44,  54,  57,  61,  62,
         63,  72,  73,  75,  90,  97,  99, 100, 101, 102, 103, 104, 105,
        107], dtype=int64),)
```

```
#  initializes an RFECV selector with a LightGBM classifier (lgbm).
# It then fits the selector to the dataset and extracts the indices of the features ranked as optimal (i.e., ran
selector = RFECV(lgbm, cv=3, n_jobs=-1)
selector = selector.fit(x, y)
lgbm_col= np.where(selector.ranking_==1)
lgbm_col
```

```
(array([  0,   1,   2,   3,   4,   5,   6,   7,  10,  11,  12,  13,  14,
         15,  16,  17,  18,  19,  20,  30,  36,  37,  38,  39,  40,  41,
         42,  43,  45,  48,  51,  56,  57,  58,  59,  60,  62,  67,  68,
         69,  74,  75,  76,  77,  78,  81,  83,  92,  93,  94,  95,  96,
         97,  98,  99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109,
        110], dtype=int64),)
```

```
#  initializes an RFECV selector with a Gradient Boosting classifier (grad).
# It then fits the selector to the dataset and extracts the indices of the features ranked as optimal (i.e., ran
selector = RFECV(grad, cv=3, n_jobs=-1)
selector = selector.fit(x, y)
grad_col= np.where(selector.ranking_==1)
grad_col
```

```
(array([  3,  17,  18,  19,  20,  37,  41,  42,  46,  57,  60,  69,  74,
         75,  76,  97,  99, 100, 102, 103, 104, 105, 106, 107, 110],
       dtype=int64),)
```

```
#  initializes an RFECV selector with an XGBoost classifier (xg).
# It then fits the selector to the dataset and extracts the indices of the features ranked as optimal (i.e., ran
selector = RFECV(xg, cv=3, n_jobs=-1)
selector = selector.fit(x, y)
xg_col= np.where(selector.ranking_==1)
xg_col
```

```
(array([  0,   1,   2,   3,   5,   6,  10,  12,  16,  17,  18,  19,  20,
         36,  37,  38,  40,  41,  42,  43,  45,  56,  57,  58,  59,  60,
         69,  74,  75,  76,  83,  93,  94,  95,  97,  98,  99, 100, 101,
        102, 103, 104, 105, 106, 107, 110], dtype=int64),)
```

```
#  generates a NumPy array containing integers from 0 to 109.
# This array represents all possible feature indices in the dataset, which can be useful for reference or for sc
np.array(list(range(0,110)))
```

```
array([  0,   1,   2,   3,   4,   5,   6,   7,   8,   9,  10,  11,  12,
        13,  14,  15,  16,  17,  18,  19,  20,  21,  22,  23,  24,  25,
        26,  27,  28,  29,  30,  31,  32,  33,  34,  35,  36,  37,  38,
        39,  40,  41,  42,  43,  44,  45,  46,  47,  48,  49,  50,  51,
        52,  53,  54,  55,  56,  57,  58,  59,  60,  61,  62,  63,  64,
        65,  66,  67,  68,  69,  70,  71,  72,  73,  74,  75,  76,  77,
        78,  79,  80,  81,  82,  83,  84,  85,  86,  87,  88,  89,  90,
```

```
         91,  92,  93,  94,  95,  96,  97,  98,  99, 100, 101, 102, 103,
        104, 105, 106, 107, 108, 109])
```

stacking with subsets of features

```
from mlxtend.feature_selection import ColumnSelector
from sklearn.pipeline import make_pipeline
```

```
#  creates a scikit-learn pipeline for the Random Forest classifier (rf).
# The ColumnSelector step selects only the features identified as optimal by the RFECV process for the Random Fo
# This ensures that the Random Forest model is trained and evaluated using only the most relevant features.
pipe_rf = make_pipeline(ColumnSelector(cols=( 0,   1,   3,  18,  19,  20,  36,  37,  40,  41,  42,  43,  45,
        46,  48,  50,  51,  52,  56,  57,  58,  59,  60,  64,  65,  67,
        69,  70,  74,  75,  76,  83,  93,  95,  97,  98,  99, 100, 101,
       102, 103, 104, 105, 106, 107)),rf)
```

```
#  creates a scikit-learn pipeline for the AdaBoost classifier (ada).
# It uses ColumnSelector to select features identified as optimal for AdaBoost (ada_col), followed by the AdaBoo
pipe_ada = make_pipeline(ColumnSelector(cols=( 17,  18,  19,  20,  37,  41,  57,  60,  75,  93,  97,  98,  99,
       100, 102, 103, 105, 106, 110)),ada)
```

```
#  creates a scikit-learn pipeline for the Extra Trees classifier (extra).
# It uses ColumnSelector to select features identified as optimal for Extra Trees (extra_col), followed by the E
pipe_extra = make_pipeline(ColumnSelector(cols=(0,   1,   3,  18,  19,  36,  37,  43,  44,  54,  57,  61,  62,
        63,  72,  73,  75,  90,  97,  99, 100, 101, 102, 103, 104, 105,
       107)),extra)
```

```
#  creates a scikit-learn pipeline for the LightGBM classifier (lgbm).
# It uses ColumnSelector to select features identified as optimal for LightGBM (lgbm_col), followed by the Light
pipe_lgbm = make_pipeline(ColumnSelector(cols=( 0,   1,   2,   3,   4,   5,   6,   7,  10,  11,  12,  13,  14,
        15,  16,  17,  18,  19,  20,  30,  36,  37,  38,  39,  40,  41,
        42,  43,  45,  48,  51,  56,  57,  58,  59,  60,  62,  67,  68,
        69,  74,  75,  76,  77,  78,  81,  83,  92,  93,  94,  95,  96,
        97,  98,  99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109,
       110)),lgbm)
```

```
#  creates a scikit-learn pipeline for the Gradient Boosting classifier (grad).
# It uses ColumnSelector to select features identified as optimal for Gradient Boosting (grad_col), followed by
pipe_grad = make_pipeline(ColumnSelector(cols=(3,  17,  18,  19,  20,  37,  41,  42,  46,  57,  60,  69,  74,
        75,  76,  97,  99, 100, 102, 103, 104, 105, 106, 107, 110)),grad)
```

```
#  creates a scikit-learn pipeline for the XGBoost classifier (xg).
# It uses ColumnSelector to select features identified as optimal for XGBoost (xg_col), followed by the XGBoost
pipe_xg = make_pipeline(ColumnSelector(cols=( 0,   1,   2,   3,   5,   6,  10,  12,  16,  17,  18,  19,  20,
        36,  37,  38,  40,  41,  42,  43,  45,  56,  57,  58,  59,  60,
        69,  74,  75,  76,  83,  93,  94,  95,  97,  98,  99, 100, 101,
       102, 103, 104, 105, 106, 107, 110)),xg)
```

```
#  creates a scikit-learn pipeline for the Multi-layer Perceptron (nn) classifier.
# It uses ColumnSelector to select all features (indices 0 to 109), followed by the MLP classifier itself.
pipe_nn = make_pipeline(ColumnSelector(cols=(0,   1,   2,   3,   4,   5,   6,   7,   8,   9,  10,  11,  12,
        13,  14,  15,  16,  17,  18,  19,  20,  21,  22,  23,  24,  25,
        26,  27,  28,  29,  30,  31,  32,  33,  34,  35,  36,  37,  38,
        39,  40,  41,  42,  43,  44,  45,  46,  47,  48,  49,  50,  51,
        52,  53,  54,  55,  56,  57,  58,  59,  60,  61,  62,  63,  64,
        65,  66,  67,  68,  69,  70,  71,  72,  73,  74,  75,  76,  77,
        78,  79,  80,  81,  82,  83,  84,  85,  86,  87,  88,  89,  90,
        91,  92,  93,  94,  95,  96,  97,  98,  99, 100, 101, 102, 103,
       104, 105, 106, 107, 108, 109)),nn)
```

```
#  re-configures the StackingCVClassifier to use the pipelines created with feature selection (sel_classifier).
# It retains the same meta-classifier (nn_meta) and 'use_probas = True' setting.
# Finally, it performs 10-fold cross-validation on this new stacking model with feature-selected base models.
sel_classifier= [pipe_rf, pipe_ada, pipe_extra, pipe_lgbm, pipe_grad, pipe_xg, pipe_nn]

nn_meta= MLPClassifier(activation = "relu", alpha = 0.001, hidden_layer_sizes = (6,12,24,12,6), random_state=10)

scl = StackingCVClassifier(classifiers= sel_classifier,
                           meta_classifier = nn_meta, # use meta-classifier
                           use_probas = True,   # use_probas = True/False
                           random_state = 10)

st_result= cross_validate(scl, x_train, y, cv = 10, scoring=scores,n_jobs=-1)
```

```
#  converts the cross-validation results from the StackingCVClassifier (st_result), which now uses feature-selec
# It then calculates and displays the mean of all the evaluation metrics across the 10 folds, providing an overa
df= pd.DataFrame(st_result)
df.mean(axis=0)
```

```
fit_time          400.310504
score_time          0.431497
test_accuracy       0.974438
test_recall         0.965347
test_precision      0.960940
test_gmean          0.972262
test_f1             0.963116
test_roc            0.972294
dtype: float64
```

Metric	Meaning	Interpretation
test_accuracy = 0.9744	Fraction of total correct predictions (both phishing and safe)	Excellent — the model correctly predicts 97.4% of samples.
test_recall = 0.9653	Of all actual *phishing* samples, how many were caught by the model	Very strong — your model catches 96.5% of phishing attacks (low false negatives).
test_precision = 0.9609	Of all samples predicted *phishing,* how many were actually phishing	High precision — only ~3.9% false alarms.
test_gmean = 0.9723	Geometric mean of sensitivity (recall) and specificity	Balanced measure — high G-Mean means the model handles both phishing and no
test_f1 = 0.9631	Harmonic mean of precision and recall	Excellent balance — the model performs well at catching phishing without too many
test_roc = 0.9723	Area under the ROC curve (AUC)	Very high (max=1.0). The model separates phishing vs. legitimate emails with ~97%

```
This Code is for phishing detection using ML models. A sophisticated framework based on Stacked Gene

                ┌──────────────────────┐
                │     Training Data    │
                └──────────┬───────────┘
                           │
                 Split using K-Fold CV
                           │
        ┌──────────────────┴──────────────────┐
        │                                     │
```

```
┌──────────────────────────────┐  ┌──────────────────────────────┐  | Training Fold |  | Validation |  | (k-1 folds) |  | Fold
│  └──────────┬───────────┘  └──────────┬───────────┘  |  |  | Train multiple models | ▼ |
┌──────────┴──────────┐  |  | Model 1 | Model 2 | Model m |
──────────┘  └──────────┬──────────┘  └──────────┬──────────┘  └──────────┬──────────┘
|  |  |  | Predictions on validation fold ▼ ▼ ▼ Pred1 Pred2 Predm
```

```
(Repeat process for all K folds)

                 │
                 ▼
    ┌──────────────────────────┐
    │ Combined Level-1 Dataset │
    │ (All model predictions)  │
    └──────────┬───────────────┘
               │
               ▼
        ┌──────────────┐
        │  Meta Model  │
        │ (Final Learner) │
        └──────┬───────┘
               ▼
        Final Prediction
          on Test Data
```